前端架构实施方案

一、方案总览

采用 B/S 混合架构，将前端系统逻辑拆分为用户表现层（前端客户服务程序）、渲染控制中台（控制与数据中台+UE 画面绘制程序） 两大核心部分，各模块通过 HTTP、WebSocket、WebRTC 协议实现解耦交互，兼顾单机演示与分布式部署需求，核心达成灾害仿真参数配置、3D 渲染画面实时展示、仿真过程交互式回放、多维数据联动可视化的前端核心功能。

二、核心架构设计

遵循**高内聚低耦合**原则，前端整体为三层逻辑架构，各层独立开发、标准化通信，即使演示阶段部署于同一物理主机，也严格遵循网络通信协议，保障架构可扩展性。

核心通信协议：

（1）HTTP：用于仿真参数提交、初始化配置、非实时数据查询；

（2）WebSocket：用于实时帧数据传输、摄像机控制指令、仿真状态同步；

（3）WebRTC：基于 UE Pixel Streaming 插件，实现 3D 渲染视频流的低延迟 P2P 传输。

三、各模块实施方案

（一）用户表现层：前端客户服务程序

1. 技术栈选型

核心技术：Vue.js/ React + ECharts（数据可视化） + Pixel Streaming SDK（UE 视频流播放） + Axios（HTTP 请求） + WebSocket 客户端

辅助工具：Vite（构建工具）、Element Plus/Ant Design（UI 组件库）、lodash（数据处理）、dayjs（时间戳处理）

2. 核心功能模块开发

本模块为用户直接操作界面，核心围绕仿真全生命周期设计功能，分为 4 大核心子模块，各模块联动实现完整交互流程。

子模块 1：仿真配置与任务提交面板

开发内容：设计可视化表单，支持用户配置地铁灾害仿真的核心参数，参数需与后端`/api/init`接口数据结构严格对齐，核心配置项包括：

（1）场景配置：场景 ID（如地铁站台、通道）、地图层级；

（2）灾害配置：灾害类型（火灾、洪水、复合灾害）、灾害起源坐标/区域、火势大小/水流速度/烟雾毒性；

（3）人群配置：总人数、初始分布区域；

（4）其他：仿真时长、计算精度。

交互逻辑：

（1）表单校验：前端做参数非空、范围校验 ，避免无效请求；

（2）任务提交：点击「开始仿真」后，通过 POST /api/init 将参数发送至控制与数据中台，前端进入「计算等待」状态；

（3）进度展示：接收中台返回的仿真计算进度，以进度条形式展示，计算完成后解锁回放功能。

子模块 2：3D 渲染与场景漫游窗口

开发内容：集成 UE 官方 webRtcPlayer.js，实现 UE 渲染视频流的嵌入与播放，作为页面核心可视化区域；开发键鼠事件捕获模块，支持场景漫游交互。

核心交互：

（1）视频流基础控制：支持视频流的全屏、暂停/继续、画质调节；

（2）摄像机漫游控制：捕获 WASD 键（移动）、鼠标拖拽（旋转视角）、滚轮（缩放）事件，将指令通过 WebSocket 实时发送至 UE 画面绘制程序，实现用户在仿真回放中自由控制观察视角；

（3）状态联动：视频流画面与仿真时间轴同步，时间轴跳转时，视频流画面瞬间刷新为对应时刻的 3D 场景。

子模块 3：高精度仿真时间轴

开发内容：作为仿真回放的核心控制组件，开发支持播放、暂停、倍速（0.5x/1x/2x/5x）、拖拽跳转、时间戳定位的高精度时间轴，时间轴刻度与仿真秒级帧数据对齐。

核心交互逻辑：

（1）播放模式：前端按选定倍速，向中台发送连续帧数据请求，中台按时间顺序推送帧数据至 UE，实现 3D 场景的动态播放；

（2）暂停/拖拽跳转模式：拖拽进度条至指定时间戳时，前端向中台发送单帧数据请求，中台检索数据库并推送对应帧数据，UE 瞬间刷新场景，实现“时间静止”观察；

（3）状态显示：时间轴上实时显示仿真状态（running/finished/paused），并标注关键事件节点，如人员开始疏散、闸机强制打开。

子模块 4：多维数据可视化侧边栏

开发内容：基于 ECharts 开发多类型图表，与仿真时间轴实时联动，实现 3D 场景与 2D 数据的可视化结合，核心图表包括：

（1）趋势类：毒烟浓度变化曲线、水位高度变化曲线、平均人员暴露时间趋势；

（2）统计类：已逃生人数/被困人数饼图、人员状态（站立/奔跑/跌倒）分布柱状图；

（3）分布类：地铁站人群密度热力图、水/烟覆盖区域地图标注。

联动逻辑：时间轴拖动至 T 时刻时，前端同步从中台获取 T 时刻的统计数据，ECharts 图表自动更新为对应时刻的数据，实现“画面看场景、图表看数据”的双重分析视角。

（二）渲染控制中台：控制与数据中台

1. 技术栈选型

核心技术：Node.js（Express/Koa） + MySQL/MongoDB（数据库） + WebSocket 服务端（ws 库） + axios（对接后端/大模型 API）

辅助工具：Sequelize/Mongoose（ORM 框架）、Redis（可选，帧数据缓存，提升回放效率）、Winston（日志记录）

2. 核心功能模块开发

本模块为前端系统的逻辑核心，承接用户表现层的指令，对接后端仿真引擎与 UE 画面绘制程序，负责数据的分发、存储、检索，核心分为 5 大子模块。

子模块 1：仿真任务调度与对接

核心工作：

（1）接收用户表现层提交的仿真参数，通过标准化接口转发至后端 Node.js 仿真服务，触发灾害仿真计算；

（2）接收后端返回的仿真计算进度、全量帧数据，实时同步至用户表现层；

（3）对接大模型 API：接收用户表现层的“预案推演”请求，截取当前仿真帧数据发送至大模型，获取决策建议（如人员疏散指引、设备操作指令），并将决策 JSON 转发至 UE 与用户表现层。

子模块 2：仿真数据缓冲与序列化存储

核心工作：

（1）接收后端仿真引擎生成的全量秒级帧数据，按”Simulation_ID（仿真任务唯一标识）和 Time_Step（时间戳）”建立双索引，对数据进行序列化处理；

（2）将结构化帧数据存入数据库（MongoDB），帧数据结构需严格遵循项目定义的 JSON 格式，包含环境数据、人群数据、事件数据、统计数据；

（3）可选：引入 Redis 做热点帧数据缓存（如最近访问的仿真帧），提升回放时的帧数据检索效率。

子模块 3：按需数据分发服务

核心工作：响应用户表现层的帧数据请求，实现**“按需推送”**，分为两种模式：

（1）连续流式发送（播放模式）：根据用户选定的倍速，按时间顺序从数据库/Redis 中提取帧数据，通过 WebSocket 连续推送至 UE 画面绘制程序；

（2）单帧发送（暂停/拖拽跳转模式）：根据用户指定的时间戳，精准检索对应帧数据，通过 WebSocket 一次性推送至 UE，实现场景瞬间刷新。

数据过滤：仅向用户表现层推送统计数据（用于可视化），避免全量帧数据传输造成前端性能损耗。

子模块 4：WebRTC 信令服务

核心工作：基于 WebRTC 协议，实现用户表现层与 UE 画面绘制程序之间的信令握手，负责视频流通道的建立、维护与断开，保障 UE Pixel Streaming 视频流的稳定、低延迟传输；

异常处理：监听视频流连接状态，当连接中断时，向前端推送提示信息，并尝试自动重连，重连失败则提供手动重连按钮。

子模块 5：仿真状态管理

核心工作：维护所有仿真任务的状态（初始化/计算中/已完成/已终止），并提

供状态查询接口；

数据管理：支持仿真任务的增删查改，以及历史仿真帧数据的批量删除/导出，避免数据库数据冗余。

（三）渲染控制中台：UE 画面绘制程序

1. 技术栈选型

核心技术：Unreal Engine 4.26 + Pixel Streaming Plugin + Blueprints（蓝图，核心逻辑开发）/ C++（高性能需求模块）

关键技术：VAT（顶点动画贴图）/ ISM（实例化静态网格体）（大规模人群渲染）、UE 粒子系统（水体/烟雾效果）、蓝图 WebSocket 客户端（对接控制与数据中台）

2. 核心功能开发要求

本模块为**无状态渲染引擎**，不依赖物理引擎的连续性，仅负责“将收到的帧数据可视化”，核心工作由 UE 开发人员完成，前端组需配合完成数据对接与指令交互的逻辑开发，核心要求如下：

（1）3D 场景搭建：基于免费 3D 模型与材质，搭建地铁站精细化场景（站台、通道、闸机、楼梯等），还原真实地铁环境；

（2）帧数据解析与可视化：

开发蓝图逻辑，解析控制与数据中台推送的 JSON 帧数据，将环境数据（水位高度、烟雾浓度/类型）、人群数据（人员坐标、朝向、状态）、事件数据（站务员动作、设备操作）转化为 UE 场景中的视觉效果；

水体模拟：采用“后端计算水位+UE 可变高度立方体+水面材质”实现，配合粒子系统实现基础水流效果；

人群渲染：使用 VAT/ISM 技术实现 500 人级别的大规模人群渲染，避免性能卡顿，根据人员状态（站立/奔跑/跌倒/溺水）切换对应动画；

（3）无状态渲染：支持帧数据的随机跳转，接收到任意时间戳的帧数据时，可瞬间刷新场景状态，无需从初始帧逐步计算；

（4）摄像机控制响应：接收用户表现层通过 WebSocket 发送的键鼠控制指令，实时调整 UE 摄像机位置与视角，实现场景漫游；

（5）视频流推流：通过 Pixel Streaming 插件，将渲染后的 3D 场景编码为 H.264 视频流，通过 WebRTC 推流至用户表现层，推流帧率不低于 30fps，保障画面流畅性；

（6）事件可视化：对大模型生成的决策事件（如闸机强制打开、站务员指引疏散）进行场景高亮标注，并显示气泡文字提示。

四、前端与其他模块对接规范

（一）与后端仿真引擎对接

（1）接口类型：HTTP POST/GET + WebSocket；

（2）核心接口：

初始化配置：`POST /api/init`，入参为仿真配置 JSON，返参为仿真任务 ID 与初始化状态；

帧数据推送：后端通过 WebSocket 向前台控制与数据中台推送全量帧数据，数据结构严格遵循项目定义；

计算进度查询：`GET /api/progress?simulationId=xxx`，返参为当前计算进度（0-100%）。

（二）与大模型 API 对接

（1）对接主体：控制与数据中台；

（2）对接逻辑：接收用户表现层的预案推演请求，截取当前帧的环境数据+人群数据，通过 HTTP POST 发送至大模型 API，获取决策 JSON 后，分别转发至 UE 与用户表现层。

（三）前端内部模块对接

（1）用户表现层 ↔ 控制与数据中台：通过 HTTP（配置提交/状态查询）、WebSocket（帧数据请求/摄像机指令）；

（2）控制与数据中台 ↔ UE 画面绘制程序：通过 WebSocket（帧数据推送/指令响应）、WebRTC 信令（视频流通道）；

（3）数据格式：所有 JSON 数据采用 UTF-8 编码，字段名使用小驼峰命名，时间戳统一为毫秒级。

五、成员分工

成员	主责工作	辅责工作
王梦可	用户表现层核心交互开发（页面搭建、功能集成、兼容性优化）	前端与控制与数据中台联调，输出前端测试问题清单
朱立	控制与数据中台服务端开发（Node.js 服务、数据库对接、接口开发）	制定全项目前端接口文档，保障对接一致性
赵涵	控制与数据中台数据层与状态管理（数据分发、大模型对接、信令异常处理）	UE 与控制与数据中台联调，验证数据与指令传输准确性
刘佳诚	UE 画面绘制程序前端侧对接开发（UE 蓝图集成、联调、可视化验证）	用户表现层全流程功能测试，反馈问题并协助优化